

1. Executive Summary

EquityFlow is a web-based startup marketplace designed to connect startup founders with potential investors. The platform allows startup owners to publish information about their projects, upload supporting materials, and manage their startup profiles through a control panel. Investors can browse available startups, analyze project information, and receive AI-generated insights that help them evaluate potential investment opportunities.

The architecture of EquityFlow is designed to support scalability, maintainability, and efficient data processing. The system follows a Client-Server architecture, where the frontend provides the user interface and communicates with a centralized backend responsible for data processing and business logic. The backend integrates AI-based analysis tools to generate insights for investors and uses a PostgreSQL database for reliable data storage and management.

By separating the presentation layer from the application logic and data storage, EquityFlow ensures that the system can evolve and scale as more users, startups, and investors join the platform.

2. Architectural Drivers

The architectural design of EquityFlow is influenced by several key functional and non-functional requirements.

Functional Requirements

- Startup owners must be able to create and manage startup profiles.
- Users must be able to upload files and supporting materials related to their startups.
- Investors must be able to browse startup projects available on the platform.
- The system must provide AI-generated insights to help investors evaluate startups.
- The platform must include a control panel for managing startup information.

Quality Attributes

Performance

The system should efficiently handle multiple users browsing startups and requesting AI insights.

Security

User accounts and startup data must be protected to prevent unauthorized access.

Scalability

The architecture should allow the system to grow as more users and startups are added.

Maintainability

The separation between frontend, backend, and data storage should make the system easier to update and extend.

3. System Overview

EquityFlow is a web-based platform that connects startup founders with investors through a centralized digital marketplace. The system enables startup owners to present their projects, upload documentation, and manage startup-related data, while investors can explore opportunities and analyze potential investments using AI-generated insights.

The platform consists of three main components:

Frontend Layer

The frontend is implemented using HTML, CSS, and JavaScript. It provides the user interface through which users interact with the system. Startup owners can upload project information and documents, while investors can browse startups and access analytical insights.

Backend Layer

The backend is implemented using Python and handles the core application logic. It processes user requests, manages startup data, generates AI-based insights for investors, and communicates with the data storage layer.

Data Layer

PostgreSQL is used as the primary relational database for the platform. It stores core system data such as user accounts, startup profiles, uploaded resource metadata, and investment records. PostgreSQL provides reliable structured storage, strong data consistency, and efficient querying for relational data.

Together, these components form a Client-Server architecture where the frontend communicates with the backend through API requests, and the backend manages interactions with the PostgreSQL database and AI services.

4. C4 Diagrams

L1 System Context Diagram

The Context Level Diagram (L1) shows the high-level EquityFlow ecosystem, detailing its interactions with users and external services. EquityFlow is a centralized startup marketplace connecting founders and investors. Founders upload project information and documentation (pitch decks, financials), while investors browse, analyze data, and make investment decisions.

Three main user types interact with the system: Startup owners publish and manage their startups; Investors evaluate startups and receive AI-generated insights; and Visitors can browse public information with limited access.

EquityFlow integrates with external services: The AI Analysis Service generates analytical insights and recommendations for investors. AWS cloud infrastructure is used for file storage and system deployment of documents. Redis serves as the primary data storage and caching layer for fast data access and session management.

Overall, EquityFlow acts as the central orchestrator managing interactions between users, the AI analysis service, cloud storage, and the internal data management system. The Context Level Diagram (L1) presents a high-level view of the EquityFlow ecosystem and illustrates how the system interacts with its main users and external services. At this level, the system is shown as a single platform that coordinates communication between users and supporting technologies without revealing internal implementation details.

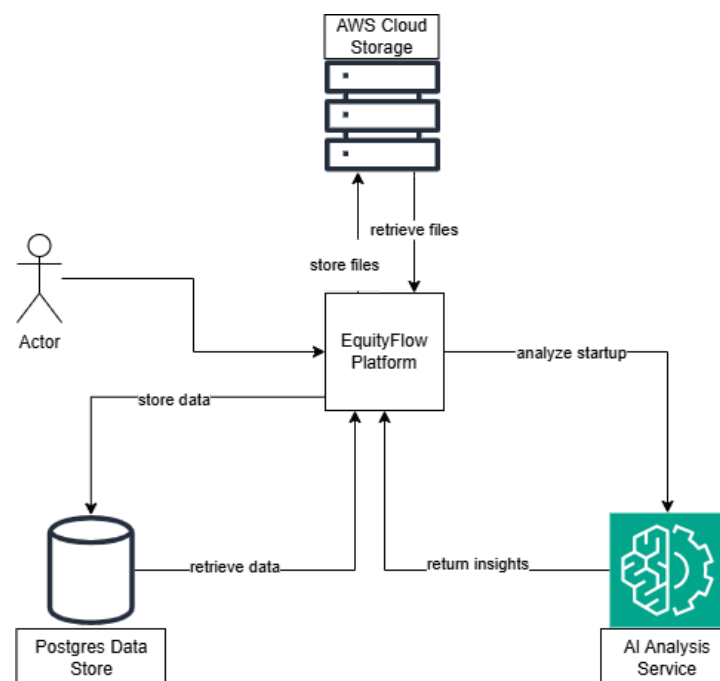
EquityFlow acts as a startup marketplace that connects startup owners with investors. Startup founders can upload information about their projects, manage their startup profiles, and provide documents such as pitch decks or other supporting materials. Investors use the platform to browse available startups, analyze project information, and make investment decisions.

Three main user groups interact with the system. Startup owners publish and manage their startup information through a control panel. Investors explore startups and receive AI-generated insights that help evaluate potential investments. Visitors can view publicly available startup information with limited access.

The system also interacts with external services. An AI Analysis Service processes startup data and generates insights such as summaries or evaluations to support investor decision-making. The platform integrates with AWS cloud services to store uploaded files and ensure reliable and scalable infrastructure.

For persistent data storage, the system uses a PostgreSQL database. PostgreSQL stores core platform data such as user accounts, startup information, and investment records while ensuring reliable and structured data management.

Overall, the Context Diagram shows EquityFlow as the central platform that manages interactions between users, AI analysis services, cloud storage infrastructure, and the PostgreSQL database



L2 Container Diagram

The L2 Container Diagram presents the high-level structure of the EquityFlow system by showing the main application containers and how they interact with each other. At this level, the architecture is divided into separate components responsible for the user interface, application logic, and data storage.

Users access the system through a web browser that loads the frontend interface built with HTML, CSS, and JavaScript. The frontend provides the visual interface where startup owners can upload startup information and documents, while investors can browse available startups and request AI-generated insights.

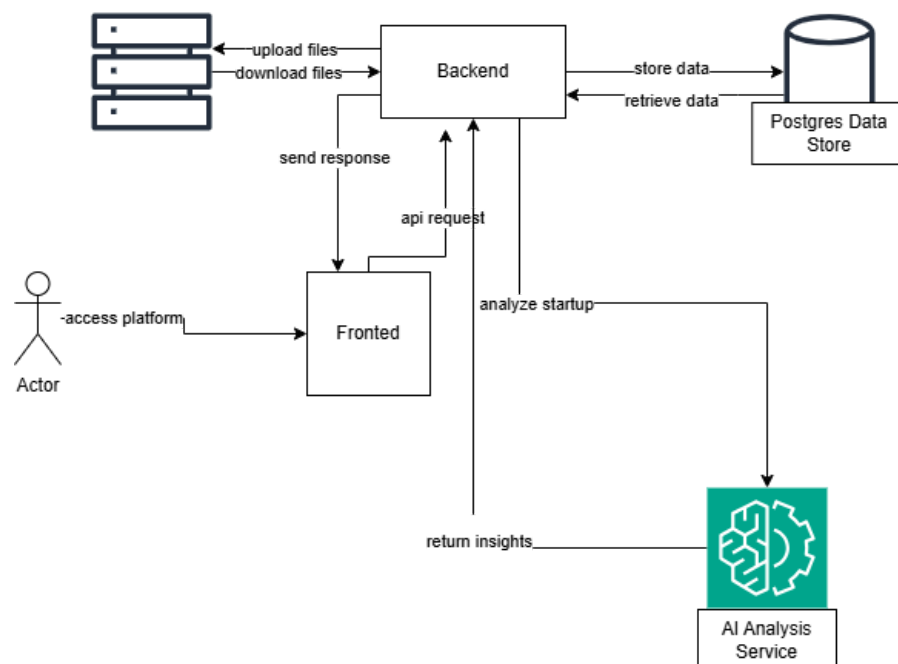
The frontend communicates with the backend server through HTTP API requests. The backend is implemented in Python and handles the core business logic of the platform. It processes user authentication, manages startup data, handles file uploads, and coordinates AI analysis for investor insights.

Startup files such as pitch decks or supporting documents are stored in AWS cloud storage. This ensures reliable and scalable storage for uploaded files. When needed, the backend retrieves these files from AWS for processing or display.

Redis is used as the primary data store and caching layer. It manages user sessions and stores frequently accessed startup data to improve performance and reduce system latency.

For AI-powered analysis, the backend communicates with an external AI service that processes startup information and returns insights that help investors evaluate investment opportunities.

Together, these containers form a client-server architecture where the frontend interacts with the backend API, and the backend coordinates communication with data storage, cloud infrastructure, and AI services.



L3 Component Diagram

The L3 Component Diagram illustrates the internal structure of the EquityFlow backend and shows how different components collaborate to process user requests and manage system functionality. While the L2 container diagram presents the main system containers, the L3 diagram focuses on the internal components inside the backend application.

The backend is implemented in Python and follows a layered architecture that separates request handling, business logic, and data access. This separation improves maintainability and makes the system easier to extend.

Transport Layer

The Transport Layer acts as the entry point for all client requests. It exposes API endpoints that receive requests from the frontend application. This layer validates incoming data and routes the requests to the appropriate services.

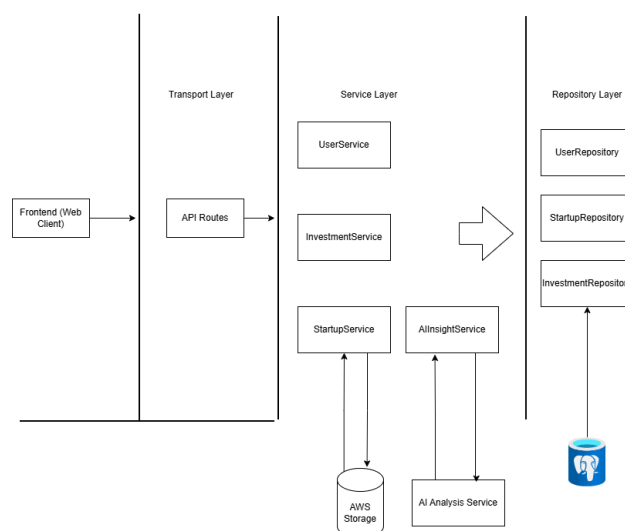
Service Layer

The Service Layer contains the core business logic of the system. Different services manage specific platform features. The UserService handles user authentication and account management. The StartupService manages startup profiles and project data uploaded by founders. The InvestmentService processes investment actions performed by investors. The AllInsightService communicates with the AI analysis service to generate insights that help investors evaluate startup opportunities.

Repository Layer

The Repository Layer is responsible for interacting with the PostgreSQL database. It performs operations such as storing user accounts, retrieving startup data, and managing investment records. By isolating database interactions in this layer, the system keeps business logic separate from data management.

Overall, the L3 diagram demonstrates how the backend processes requests from the frontend, applies business logic through service components, and stores or retrieves data from the PostgreSQL database while coordinating AI-based analysis when needed.



5. Technology Selection & Justification

1. Backend Framework: FastAPI / Django

Selected: FastAPI and Django are used for the backend implementation of the EquityFlow platform.

Justification: Python frameworks were chosen because they provide strong support for building scalable web applications and integrating AI services. FastAPI offers high performance and efficient API development, making it suitable for handling asynchronous requests and communication with AI services. Django provides a robust structure for managing application logic, authentication, and database operations. Together, they enable the backend to manage user accounts, startup data, and investment functionality efficiently.

Alternatives Considered: Flask was considered due to its simplicity; however, FastAPI and Django were preferred because they provide better scalability, built-in features, and stronger support for larger applications.

2. Database: PostgreSQL

Selected: PostgreSQL is used as the primary relational database for EquityFlow.

Justification: PostgreSQL provides reliable storage for structured data such as user accounts, startup profiles, investment records, and system metadata. It supports strong data consistency, relational queries, and scalability, making it well suited for applications that manage structured relationships between users, startups, and investments.

Alternatives Considered: NoSQL databases such as MongoDB were considered; however, PostgreSQL was chosen because the platform requires structured relational data and strong transactional integrity.

3. File Storage: AWS S3

Selected: AWS S3 is used to store uploaded files and startup-related documents.

Justification: Startup owners may upload pitch decks, documents, and other materials related to their projects. AWS S3 provides reliable and scalable cloud storage for these files. It allows the system to store large files securely and retrieve them efficiently when needed.

Alternatives Considered: Local server storage was considered but rejected because cloud storage provides better scalability, durability, and accessibility.

4. AI Analysis Service: Google Gemini

Selected: Google Gemini is used as the AI service for generating startup analysis and insights.

Justification: Gemini enables the platform to process startup descriptions and generate insights that help investors evaluate potential opportunities. AI analysis can include summaries, evaluations, and other intelligent insights that improve decision-making for investors.

Alternatives Considered: Other AI services such as OpenAI GPT models were considered; however, Gemini was selected for its strong natural language processing capabilities and integration options.

5. Frontend Framework: React

Selected: React is used for the frontend user interface.

Justification: React provides a component-based architecture that simplifies the development of complex user interfaces. It allows the platform to create interactive dashboards where startup owners manage their projects and investors explore investment opportunities.

Alternatives Considered: Traditional HTML, CSS, and JavaScript were considered, but React was selected because it improves scalability, maintainability, and user experience in larger web applications.

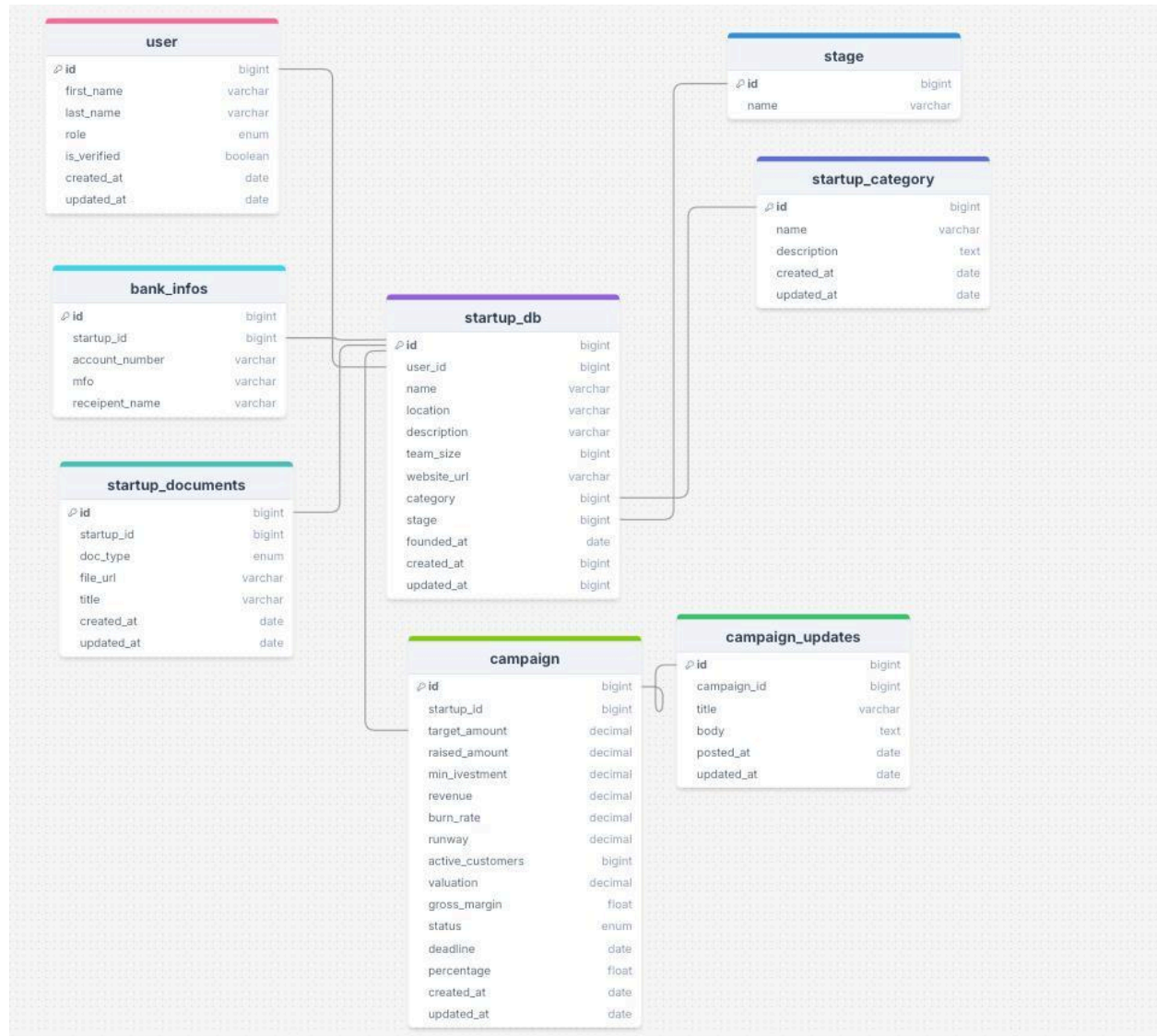
6. Authentication: JWT (JSON Web Tokens)

Selected: JWT is used for authentication and user authorization.

Justification: JWT allows secure communication between the React frontend and the Python backend. It enables stateless authentication, which improves scalability and simplifies API security. Each authenticated user receives a token that verifies their identity when accessing protected endpoints.

Alternatives Considered: Session-based authentication was considered; however, JWT was selected because it is more suitable for modern API-based architectures.

DATA MODEL



Entity-Relationship Data Model Description

The EquityFlow data model is designed to manage startup information, user accounts, investment campaigns, and supporting documents within a structured relational database. The model ensures data consistency and supports relationships between startups, founders, investors, and funding campaigns. PostgreSQL is used as the primary database to store and manage these entities while maintaining strong relational integrity.

The data model organizes the platform into several main groups: user management, startup management, campaign management, and supporting metadata such as categories and stages.

1. User Management

- User

The User entity represents individuals who interact with the platform. Each user may have different roles such as startup owner, investor, or visitor. The table stores user profile information including first name, last name, role, verification status, and timestamps for account creation and updates.

2. Startup Management

- Startup

The Startup entity represents startup projects registered on the platform. Each startup is associated with a user who acts as the startup owner. The table contains information such as startup name, location, description, team size, website URL, category, stage, and founding date.

- Startup Documents

The Startup Documents entity stores documents uploaded by startup owners, such as pitch decks or financial reports. Each document is linked to a specific startup and includes metadata such as document type, file URL, title, and creation timestamps.

- Bank Infos

The Bank Infos entity stores financial account details related to a startup. This information is required for managing investment transactions and transferring funds raised through the platform. Each record is associated with a specific startup.

3. Campaign Management

- Campaign

The Campaign entity represents investment campaigns created by startups to raise funds. Each campaign belongs to a specific startup and includes financial metrics such as target funding amount, raised amount, minimum investment, valuation, revenue, burn rate, and active customers. The campaign also tracks status, deadline, and percentage of funding completed.

- Campaign Updates

The Campaign Updates entity stores updates published during a campaign. Startup founders can use this feature to inform investors about progress, milestones, or important announcements related to the campaign.

4. Startup Classification

- Startup Category

The Startup Category entity classifies startups into different industry groups such as technology, healthcare, or fintech. Each startup is assigned to a category, which helps investors filter and explore startups based on their interests.

- Stage

The Stage entity represents the development stage of a startup, such as idea stage, prototype stage, early traction, or growth stage. This helps investors understand the maturity level of each startup.

Entity Relationships

The EquityFlow data model contains several important relationships between entities:

- User–Startup Relationship (One-to-Many)

A single user can create and manage multiple startups, but each startup is associated with one primary user.

- Startup–Documents Relationship (One-to-Many)

A startup may have multiple uploaded documents, such as pitch decks, business plans, and financial reports.

- Startup–Bank Information Relationship (One-to-One)

Each startup can have one bank information record used for handling financial transactions.

- Startup–Campaign Relationship (One-to-Many)

A startup can create multiple funding campaigns over time.

- Campaign–Updates Relationship (One-to-Many)

Each campaign can contain multiple updates that inform investors about campaign progress.

- Startup–Category Relationship (Many-to-One)

Multiple startups can belong to the same category.

- Startup–Stage Relationship (Many-to-One)

Multiple startups can be assigned to the same development stage.

Data Normalization

The EquityFlow database schema follows relational database normalization principles and is structured up to the **Third Normal Form (3NF)** to ensure efficient data organization and integrity. Each entity contains a primary key that uniquely identifies records, and all attributes depend directly on that key.

Data is separated into multiple related tables such as users, startups, campaigns, documents, categories, and stages in order to eliminate redundancy and avoid data anomalies. Relationships between these entities are maintained through foreign keys, which ensures consistent connections between startup owners, their startups, investment campaigns, and related documents.

By organizing the schema in Third Normal Form, the system prevents duplicate data storage, simplifies database maintenance, and improves query efficiency within the PostgreSQL database. This structure also allows the platform to scale as more users, startups, and investment campaigns are added to the system.

Authentication Routes

Endpoint

POST /api/auth/register

Description

Registers a new user account in the system. The user selects a role (startup owner or investor) during registration.

Authentication

Public

Request Body

```
{  
  "email": "string",  
  "password": "string",  
  "first_name": "string",  
  "last_name": "string",  
  "role": "string"  
}
```

Response

201 Created – User successfully registered

Endpoint

POST /api/auth/login

Description

Authenticates a user and returns a JWT token that is used to access protected endpoints.

Authentication

Public

Request Body

```
{  
  "email": "string",  
  "password": "string"  
}
```

Response

200 OK – Returns JWT authentication token

Endpoint

POST /api/auth/send-otp

Description

Sends a one-time password (OTP) to the user's email for verification purposes.

Authentication

Public

Request Body

```
{  
  "email": "string"  
}
```

Response

200 OK – OTP sent successfully

Endpoint

POST /api/auth/verify-otp

Description

Verifies the OTP code sent to the user's email address.

Authentication

Public

Request Body

```
{  
  
  "email": "string",  
  
  "otp": "string"  
}
```

Response

200 OK – OTP successfully verified

Startup Management Routes

Endpoint

POST /api/startup/create

Description

Creates a new startup profile for a registered startup owner.

Authentication

Bearer Token Required

Request Body

```
{  
  
  "name": "string",  
  
  "description": "string",  
  
  "location": "string",  
  
  "website_url": "string",  
  
  "team_size": 0,
```

```
"category_id": 0,  
"stage_id": 0,  
"founded_at": "string"  
}
```

Response

201 Created – Startup successfully created

Campaign Management Routes

Endpoint

POST /api/startup/campaign

Description

Creates a new investment campaign for a startup.

Authentication

Bearer Token Required

Request Body

```
{  
  "startup_id": 0,  
  "target_amount": 0,  
  "min_investment": 0,  
  "revenue": 0,  
  "revenue_share": 0,  
  "burn_rate": 0,  
  "runway": 0,  
  "active_customers": 0,  
  "evaluation": 0,  
  "gross_margin": 0,  
  "status": "string",  
  "deadline": "string"
```

```
}
```

Response

201 Created – Campaign successfully created

Endpoint

POST /api/startup/campaign/update

Description

Publishes updates related to an active investment campaign. Startup owners can post updates to inform investors about progress or milestones.

Authentication

Bearer Token Required

Request Body

```
{
```

```
"campaign_id": 0,
```

```
"title": "string",
```

```
"body": "string"
```

```
}
```

Response

201 Created – Campaign update posted

Financial Information Routes

Endpoint

POST /api/startup/bank-info

Description

Stores bank account information associated with a startup. This information is required for managing investment transactions and fund transfers.

Authentication

Bearer Token Required

Request Body

```
{
```

```
"startup_id": 0,  
"mfo": "string",  
"account_number": "string",  
"recipient_name": "string"  
}
```

Response

201 Created – Bank information stored successfully

Quality Attribute Scenarios

Performance

Scenario: An investor browses the platform during peak usage hours and requests AI analysis for several startups.

Response: The backend processes the request through the API and sends the startup data to the AI analysis service. The system retrieves relevant startup information from the PostgreSQL database and returns AI-generated insights to the investor.

Measure: The platform should respond to normal browsing requests within 1–2 seconds, while AI insight generation should be completed within approximately 5–10 seconds depending on the complexity of the analysis.

Security

Scenario: An unauthorized user attempts to access protected startup data or perform actions without authentication.

Response: The backend verifies the user's authentication token using JWT. If the user is not authenticated or does not have the required permissions, the request is rejected.

Measure: The system returns a 401 Unauthorized or 403 Forbidden response, preventing access to protected resources and ensuring that only authorized users can access startup and investment data.

Availability

Scenario: A user attempts to access the platform while the system is experiencing high traffic or temporary service interruptions.

Response: The backend continues serving user requests through the API while maintaining access to essential data stored in the PostgreSQL database and files stored in AWS S3.

Measure: The system should maintain availability for core platform features such as browsing startups and viewing startup profiles, ensuring minimal disruption for users during high system load.

Risks & Mitigation Strategies

AI Processing Latency

Impact: AI analysis of startup data may take several seconds depending on the complexity of the request. This could create delays for investors waiting for insights.

Mitigation Strategy: The system sends startup data to the Gemini AI service through the backend and processes the response asynchronously when possible. This prevents the user interface from freezing and allows the platform to notify users when analysis results are ready.

High System Load

Impact: A large number of users browsing startups or requesting AI insights simultaneously could increase server load and slow down the system.

Mitigation Strategy: The platform uses a scalable architecture with a Python backend and PostgreSQL database. The system can be deployed on cloud infrastructure such as AWS, allowing resources to be scaled when traffic increases.

File Storage Growth

Impact: Startup owners may upload many documents such as pitch decks, reports, or supporting materials, which could increase storage usage.

Mitigation Strategy: Files are stored in AWS S3, which provides scalable cloud storage. This prevents the application server from being overloaded with file management tasks and ensures reliable long-term storage.

Unauthorized Access

Impact: Malicious users may attempt to access private startup data or investor actions without proper authorization.

Mitigation Strategy: The system uses JWT-based authentication and role-based access control. Each request is validated by the backend to ensure that only authorized users can access or modify protected resources.

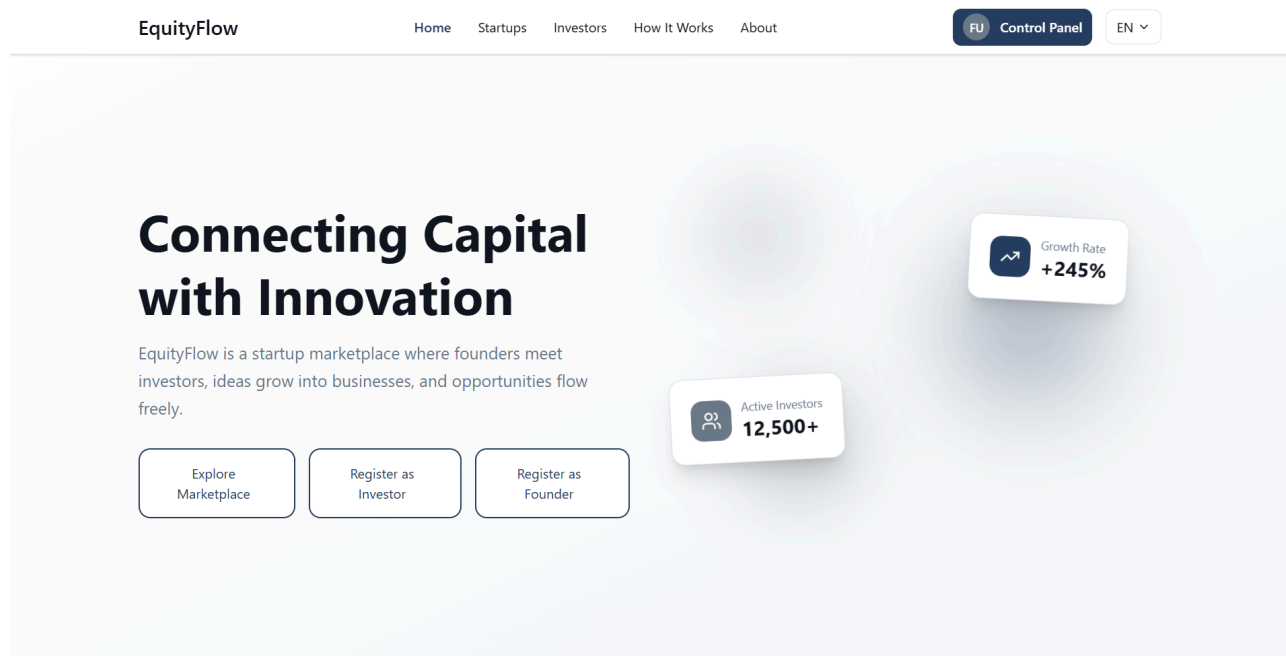
Database Reliability

Impact: Database failures or data corruption could affect the availability of startup information and investment records.

Mitigation Strategy: PostgreSQL provides reliable relational data management with transaction support. Regular backups and cloud-based database hosting can be used to ensure data integrity and recovery if failures occur.

UI/UX demos

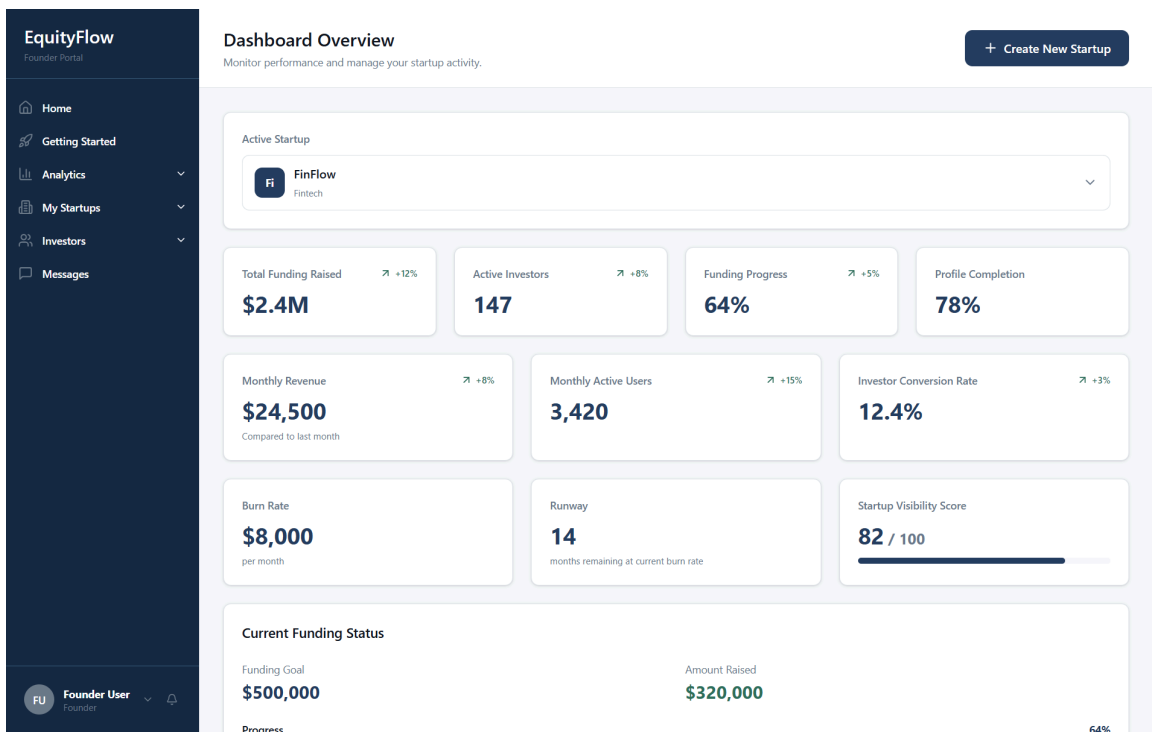
Home page



Who We Serve

Whether you're looking to invest, raise capital, or simply explore, EquityFlow

Main dashboard



Startup page for visitors

EquityFlow

[Home](#)[Startups](#)[Investors](#)[How It Works](#)[About](#)

FUControl Panel

EN

FF

FinFlow

Fintech

Series A

San Francisco, CA

REVENUE

\$42,500

/ month

+12% MoM

BURN RATE

\$8,000

/ month

Stable

RUNWAY

14

months

Healthy

INVESTOR INTEREST

147

investors

Trending

ACTIVE CUSTOMERS

2,840

paying users

+24% growth

VALUATION

\$8.5M

pre-money

Series A

TEAM SIZE

24

employees

Scaling

GROSS MARGIN

68%

profitability

Strong

Funding Goal

\$500,000

Amount Raised

\$320,000

64% funded

23 days left

Minimum Investment

\$1,000

Invest Now

Investment involves risk. Please read our risk disclosure.

AI Investment Insight

Machine-assisted risk and growth evaluation based on financial, traction, and market signals.

AI Confidence Score

82 / 100

Moderate-High Potential

Risk Level

Medium

Balanced Risk Profile

Predicted 12-Month Growth

+18–25%

Growth Trajectory

Our model indicates steady growth driven by recurring revenue and strong customer retention. Market conditions remain favorable with increasing demand in the fintech sector.

View Full AI Analysis

Disclaimer: AI insights are predictive estimates based on available data and do not guarantee investment outcomes. Please conduct your own due diligence and consult with financial advisors before making investment decisions.